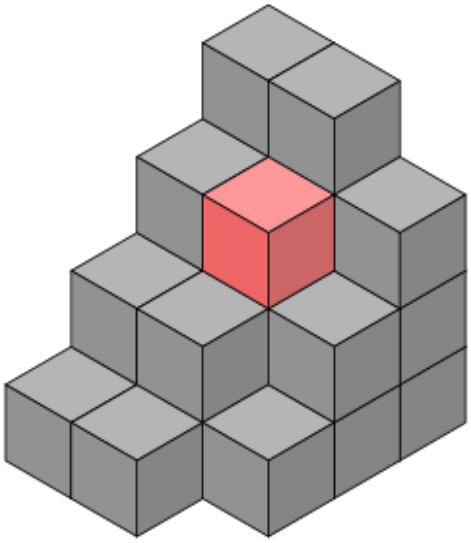
Laporan Tugas Kecil 2

IF2211 STRATEGI ALGORITMA

Voxelization Objek 3D menggunakan Octree



Disusun oleh:

Tengku Naufal Saqib

NIM 13524012

Program Studi Teknik Informatika

Institut Teknologi Bandung

Februari 2026

1 Pendahuluan

1.1 Latar Belakang

Dalam pengolahan grafika komputer dan representasi objek tiga dimensi, salah satu pendekatan yang umum digunakan adalah representasi berbasis voxel. Voxel (volumetric pixel) merupakan representasi diskret dari ruang tiga dimensi yang membagi objek menjadi kubus-kubus kecil berukuran seragam. Representasi ini banyak digunakan dalam berbagai bidang seperti pengembangan game, simulasi fisika, pemrosesan citra medis, dan visualisasi data 3D.

Proses konversi objek 3D dari bentuk mesh (seperti file .obj) menjadi voxel dikenal sebagai voxelization. Proses ini tidak trivial karena melibatkan penentuan bagian mana dari ruang yang berinteraksi dengan permukaan objek. Oleh karena itu, diperlukan pendekatan yang efisien untuk membagi ruang dan menentukan bagian yang relevan.

Salah satu struktur data yang efektif untuk menangani pembagian ruang tiga dimensi adalah Octree. Octree merupakan struktur tree di mana setiap node memiliki delapan anak, sehingga memungkinkan pembagian ruang secara rekursif menjadi delapan bagian yang lebih kecil. Dengan menggunakan Octree, proses voxelization dapat dilakukan secara efisien dengan hanya memproses bagian ruang yang berpotensi berinteraksi dengan objek.

Dalam tugas ini, pendekatan divide and conquer digunakan untuk mengimplementasikan voxelization. Ruang dibagi secara rekursif hingga mencapai kedalaman tertentu, kemudian bagian yang relevan dijadikan voxel. Pendekatan ini memungkinkan efisiensi dalam waktu dan penggunaan memori dibandingkan dengan pembagian ruang secara brute-force.

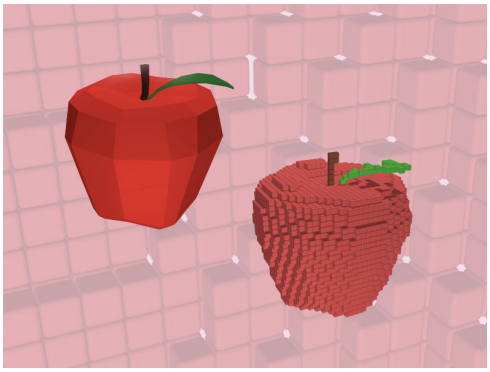
1.2 Tujuan

1. Mengimplementasikan program yang dapat melakukan voxelization terhadap objek tiga dimensi dari file .obj.
2. Menerapkan algoritma divide and conquer menggunakan struktur data Octree.
3. Menghasilkan representasi voxel dalam bentuk file .obj.
4. Menampilkan informasi statistik terkait proses voxelization seperti jumlah voxel, jumlah node Octree, dan waktu eksekusi program.

2 Dasar Teori

2.1 Voxelization

Vokselisasi adalah proses konversi (aproksimasi) objek tiga dimensi kontinu (umumnya berupa poligon) menjadi representasi diskrit. Jika dalam dua dimensi proses serupa disebut pikselisasi yang menghasilkan pixel, vokselisasi menghasilkan voxel (volumetric pixel). Setiap voxel mewakili sebuah volume kecil berbentuk kubus di dalam ruang tiga dimensi. Secara praktis, proses ini dilakukan dengan membagi ruang menjadi kubus-kubus kecil. Jika sebuah kubus berinteraksi atau berpotongan dengan permukaan objek, kubus tersebut akan diikutsertakan sebagai bagian dari representasi voxel objek tersebut.



Gambar 1: Hasil perubahan objek tiga dimensi menjadi voxel

Algoritma vokselisasi dapat diklasifikasikan menurut target objek geometris primitifnya (Aleksandrov, dkk):

1. Titik, Garis, dan Kurva: Menggunakan algoritma yang memprioritaskan tingkat konektivitas spesifik untuk memastikan tidak ada titik yang terputus.

2. Vokselisasi Permukaan (Surface Voxelization): Proses ini hanya memvokselisasi batas luar (boundary) atau kulit objek. Aleksandrov menyoroti bahwa pendekatan ini sering kali lebih ideal karena prosesnya jauh lebih cepat dan hemat memori (memory footprint yang kecil), terutama jika voxel disimpan secara renggang (sparse).

3. Vokselisasi Benda Padat (Solid Voxelization): Berbeda dengan vokselisasi permukaan, metode ini merepresentasikan seluruh struktur benda hingga ke bagian paling dalamnya.

Agar vokselisasi dan manipulasi voxel berjalan efisien tanpa membebani memori komputer secara berlebihan, diperlukan struktur data khusus. Struktur data ini dibagi berdasarkan seberapa sering data harus diperbarui (update frequency):

1. Grid Statis (Static Grids): Ideal untuk lingkungan objek 3D yang diam (statis). Kategori ini didominasi oleh metode berbasis SVO (Sparse Voxel Octree) yang berfokus kuat pada kompresi ukuran memori dan pelestarian atribut objek.

2. Grid Dinamis (Dynamic Grids): Dirancang untuk skenario simulasi di mana objek sering berubah atau bergerak secara real-time.

Berkat efisiensinya dalam pemrosesan keruangan diskrit, representasi berbasis voxel kini diimplementasikan secara luas di berbagai domain seperti pada bidang grafika komputer, dimanfaatkan untuk proses rendering (fast ray tracing), analisis bayangan, analisis visibilitas, dan deteksi benturan objek (collision detection). Dalam dunia medis digunakan sebagai representasi spasial pada perangkat lunak pemrosesan hasil CT Scan maupun MRI, sehingga memungkinkan dokter untuk menginvestigasi organ dalam bentuk 3 dimensi. Selain itu digunakan dalam pemodelan Geospasial dan Perkotaan.

2.2 Octree

Octree adalah struktur data tree yang digunakan untuk merepresentasikan pembagian ruang tiga dimensi. Setiap node dalam Octree merepresentasikan sebuah kubus dan memiliki maksimal delapan

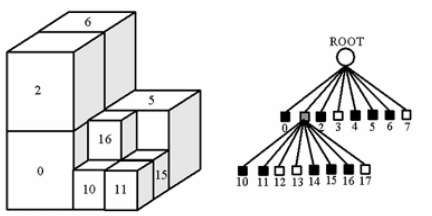
anak (octants), yang merupakan hasil pembagian kubus tersebut menjadi delapan bagian yang lebih kecil.

Proses kerja Octree:

1. Dimulai dari sebuah kubus besar (root).
2. Jika diperlukan, kubus dibagi menjadi delapan sub-kubus.
3. Setiap sub-kubus dapat dibagi kembali secara rekursif.
4. Proses berhenti ketika mencapai kondisi tertentu (misalnya kedalaman maksimum).

Keuntungan penggunaan Octree:

- Mengurangi jumlah ruang yang perlu diproses
- Efisien untuk pencarian dan representasi ruang
- Cocok untuk algoritma berbasis divide and conquer



Gambar 2: Visualisasi Octree terhadap 3D Model.

2.3 Divide and Conquer

Divide and conquer adalah paradigma algoritma yang bekerja dengan membagi masalah besar menjadi sub-masalah yang lebih kecil, menyelesaikan masing-masing sub-masalah, kemudian menggabungkan hasilnya.

Tahapan divide and conquer:

1. Divide: membagi persoalan menjadi beberapa upa-persoalan yang memiliki kemiripan dengan persoalan semula namun berukuran lebih kecil (idealnya setiap upa-persoalan berukuran hampir sama)
2. Conquer: menyelesaikan (solve) masing-masing upa-persoalan (secara langsung jika sudah berukuran kecil atau secara rekursif jika masih berukuran besar)
3. Combine: menggabungkan solusi masing-masing upa-persoalan sehingga membentuk solusi persoalan semula.

3 Perancangan Algoritma

3.1 Alur Program Utama

Program yang dikembangkan bertujuan untuk mengubah model tiga dimensi dalam format .obj menjadi representasi voxel menggunakan struktur data Octree dan pendekatan divide and conquer.

Secara umum, alur solusi yang diimplementasikan adalah sebagai berikut:

1. Membaca file .obj dan menyimpan data vertex serta face ke dalam struktur data mesh.
2. Menentukan bounding cube dari seluruh objek sebagai root dari Octree.
3. Melakukan subdivisi ruang secara rekursif menggunakan Octree.
4. Menentukan apakah setiap cube berinteraksi dengan objek menggunakan metode pengecekan intersection.

5. Menghasilkan voxel dari leaf node pada kedalaman maksimum.
6. Mengekspor hasil voxelization ke dalam file .obj.
7. Menampilkan statistik hasil proses pada command line.

3.2 Algoritma Divide and Conquer

Algoritma voxelization pada program ini dirancang menggunakan paradigma divide and conquer. Ide utamanya adalah bahwa ruang tiga dimensi yang memuat objek tidak langsung diproses seluruhnya dalam resolusi tinggi, melainkan dibagi secara bertahap menjadi ruang-ruang yang lebih kecil. Dengan demikian, program hanya memproses bagian ruang yang benar-benar relevan terhadap objek, sedangkan bagian yang kosong dapat langsung diabaikan. Pendekatan ini membuat proses menjadi lebih efisien dibandingkan pemeriksaan brute-force pada seluruh ruang.

3.2.1 Representasi Awal Ruang

Sebelum proses rekursif dimulai, program terlebih dahulu membaca seluruh vertex dari mesh objek, lalu menentukan bounding cube, yaitu kubus terkecil yang dapat mencakup seluruh objek. Bounding cube ini menjadi akar dari Octree.

Cube awal ini merepresentasikan seluruh ruang kerja yang akan dianalisis. Karena Octree bekerja pada pembagian kubus menjadi delapan bagian yang sama besar, maka bounding box biasa diperluas menjadi bounding cube, sehingga panjang sisi pada sumbu x, y, dan z seragam.

3.2.2 Tahap Divide

Pada tahap divide, sebuah cube dibagi menjadi delapan sub-cube yang ukurannya masing-masing setengah dari parent cube. Pembagian ini dilakukan karena pada tahap awal, ukuran cube masih terlalu besar untuk merepresentasikan detail permukaan objek secara akurat.

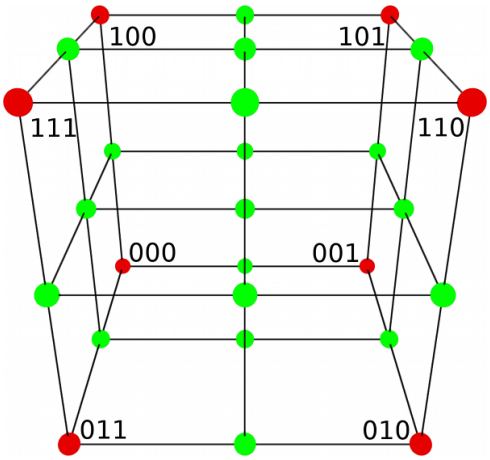
Secara geometris, jika sebuah cube memiliki:

- titik minimum (xmin, ymin, zmin)
- panjang sisi s

maka setelah dibagi, akan diperoleh 8 child cube dengan:

- panjang sisi s/2
- titik minimum yang diperoleh dengan menambahkan kombinasi offset 0 atau s/2 pada masing-masing sumbu

Dengan kata lain, setiap cube dibelah pada titik tengah sumbu x, y, dan z, sehingga menghasilkan 8 oktan ruang.



Gambar 3: Visualisasi Pemisahan 1 Kubus menjadi Octree terhadap 3D Model.

Tahap divide tidak selalu dilakukan pada setiap cube. Cube hanya dibagi lagi jika cube tersebut masih relevan terhadap objek, dan kedalamannya belum mencapai maxDepth. Jadi, pembagian ruang dilakukan secara selektif, bukan menyeluruh.

3.2.3 Tahap Conquer

Tahap conquer adalah inti pengambilan keputusan pada setiap node Octree. Pada tahap ini, program menentukan apakah sebuah cube perlu:

- dibuang,
- dibagi lagi,
- atau disimpan sebagai voxel.

Untuk itu, dilakukan proses klasifikasiCube(mesh, cube), yaitu klasifikasi status cube terhadap objek.

1. Cube di luar objek

   Jika cube sama sekali tidak berinteraksi dengan mesh, maka cube tersebut diberi status OUTSIDE. Artinya, ruang ini kosong dan tidak mengandung bagian permukaan objek.

   Karena cube ini tidak relevan, maka rekursi berhenti pada node tersebut. Inilah bentuk efisiensi utama dari divide and conquer: submasalah yang tidak penting langsung dieliminasi tanpa perlu dibagi lebih lanjut.

2. Cube berinteraksi dengan objek

   Jika cube berpotongan dengan permukaan objek, maka cube dianggap INTERSECTING. Artinya, ada kemungkinan bahwa sebagian ruang cube ditempati oleh permukaan mesh.

   Jika kondisi ini terjadi, ada dua kemungkinan:

   - Jika depth saat ini belum mencapai maxDepth, cube akan dibagi lagi menjadi delapan child cube, lalu masing-masing child diproses secara rekursif.
   - Jika depth saat ini sudah sama dengan maxDepth, maka cube tidak dibagi lagi dan langsung dianggap sebagai voxel hasil akhir.

Dengan mekanisme ini, semakin dekat cube terhadap detail permukaan objek, semakin halus pembagiannya, sampai mencapai resolusi maksimum yang ditentukan pengguna.

3.2.4 Tahap Combine

Tahap combine pada algoritma ini tidak berupa penggabungan numerik seperti pada merge sort, melainkan berupa pengumpulan seluruh voxel hasil dari leaf node yang valid.

Setelah proses rekursi selesai, seluruh cube pada depth maksimum yang dinyatakan berinteraksi dengan objek dikumpulkan ke dalam sebuah daftar voxel. Daftar inilah yang kemudian digunakan untuk:

- dihitung statistiknya,
- dikonversi menjadi geometri kubus,
- diekspor ke file .obj.

Dengan demikian, hasil akhir dari proses divide and conquer adalah sekumpulan voxel uniform yang merepresentasikan permukaan objek.

3.3 Struktur Data

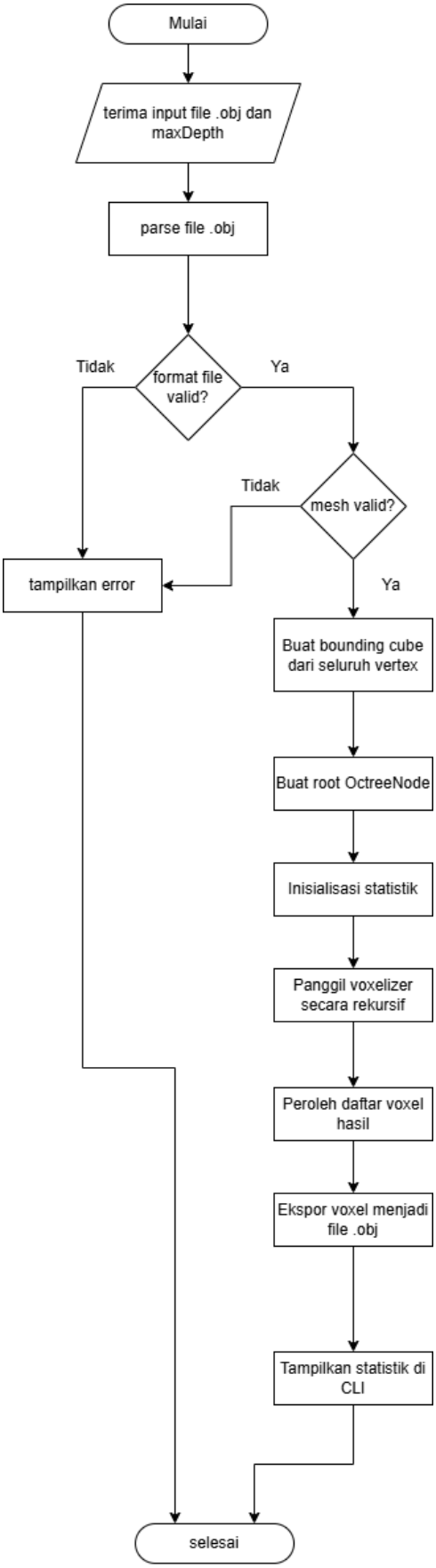
No	Struktur Data	Atribut	Tipe Data	Deskripsi
1	Vertex	x, y, z	double	Merepresentasikan titik dalam ruang tiga dimensi
2	Face	a, b, c	int	Menyimpan indeks vertex yang membentuk satu segitiga
3	Mesh	vertices, faces	List<Vertex>, List<Face>	Menyimpan seluruh model objek 3D
4	Cube	min, size	Vertex, double	Merepresentasikan kubus dengan titik minimum dan panjang sisi
5	OctreeNode	bounds, depth, children, isLeaf	Cube, int, OctreeNode[8], boolean	Node dalam struktur Octree
6	Voxel	bounds	Cube	Merepresentasikan satu voxel (kubus kecil hasil akhir)
7	Stats	nodesPerDepth, prunedPerDepth, maxObservedDepth	Map<Integer, Integer>, int	Menyimpan statistik proses voxelization

Tabel 1: Struktur Data untuk Voxelization

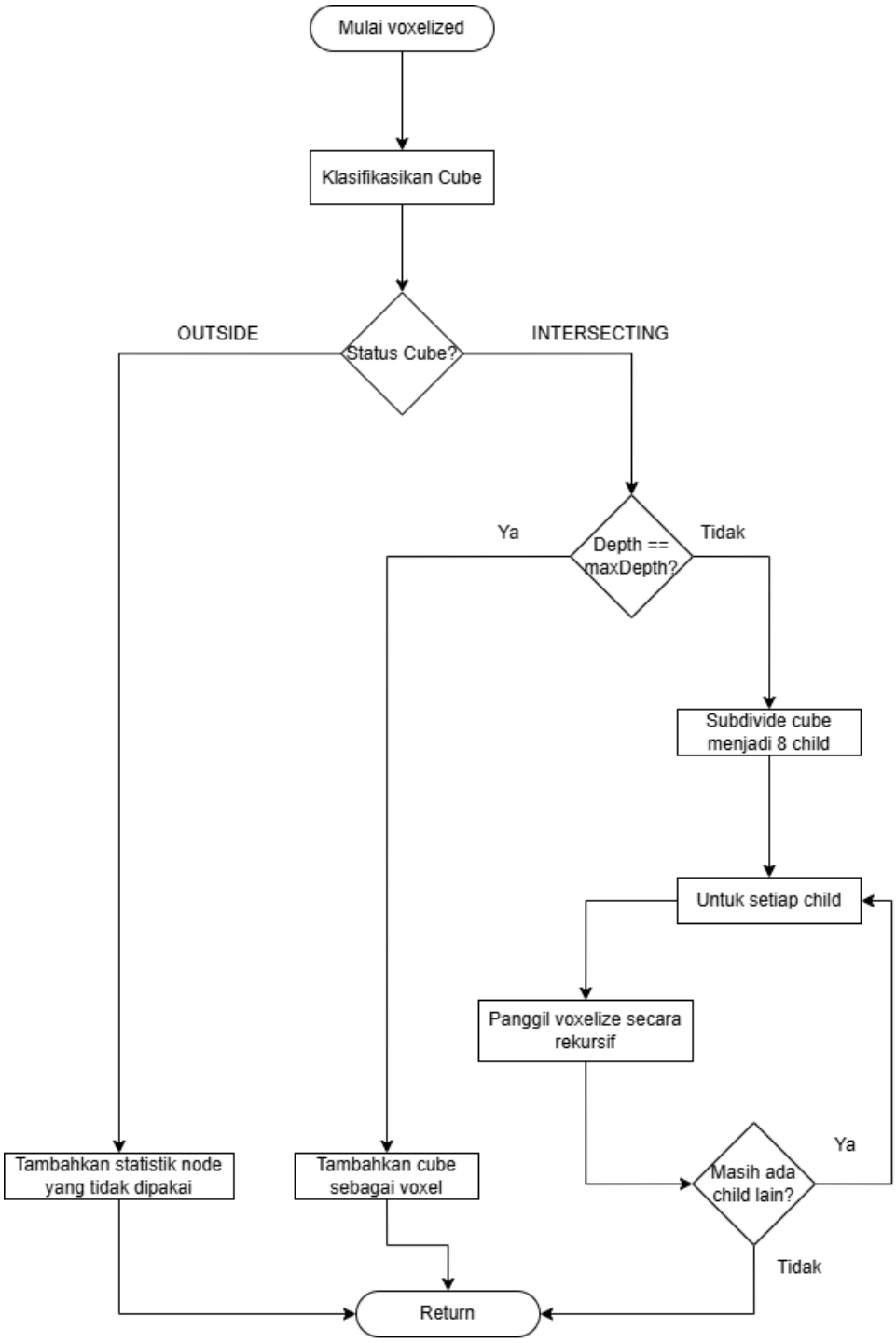
3.3.1 Pseudocode

```
Voxelize(mesh, node, maxDepth, voxelList):

    status = classifyCube(mesh, node.bounds)

    if status == OUTSIDE:
        return

    if node.depth == maxDepth:
        tambahkan node.bounds ke voxelList            //combine
        return

    subdivide node menjadi 8 child                   //divide

    for setiap child pada node:
        Voxelize(mesh, child, maxDepth, voxelList)  //conquer
```

3.4 Flowchart



Gambar 4: Flowchart algoritma keseluruhan.



Gambar 5: Flowchart algoritma divide and conquer pada voxelizer.

4 Hasil dan Pengujian

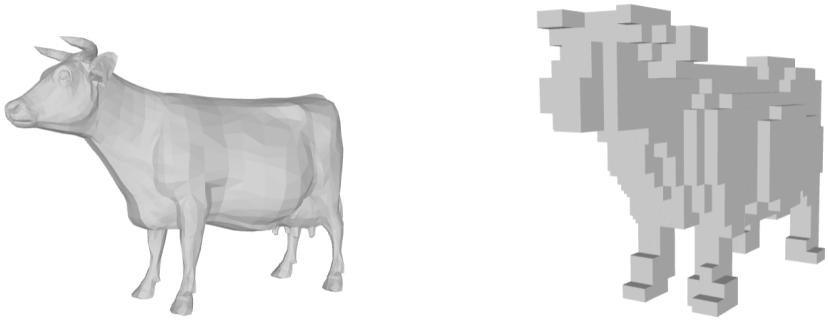
4.1 Skenario Pengujian

4.1.1 Pengujian Black Box

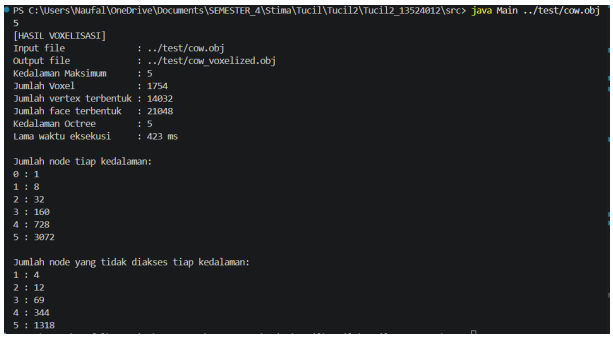
Tabel 2: Skenario Pengujian Blackbox

Kode	Skenario	Input	Expected Output	Status
TC-01	Input valid	cow.obj, 5	Output .obj terbentuk dan statistik tampil	Pass
TC-02	File tidak ditemukan	tidak_ada.obj, 4	Error file tidak ditemukan	Pass
TC-03	Depth bukan integer	line.obj, abc	Error depth harus integer	Pass
TC-04	Depth negatif	line.obj, -1	Error depth tidak valid	Pass
TC-05	File kosong	kosong.obj, 3	Error mesh tidak memiliki data	Pass
TC-06	Tanpa face	hanya_vertex.obj, 3	Error mesh tidak memiliki data	Pass
TC-07	Format file salah	teapot.obj, 3	Error format vertex tidak valid	Pass
TC-08	Pengaruh depth	pumpkin.obj, 3 / 6	Depth lebih besar menghasilkan voxel lebih detail	Pass

4.2 Tangkapan Layar

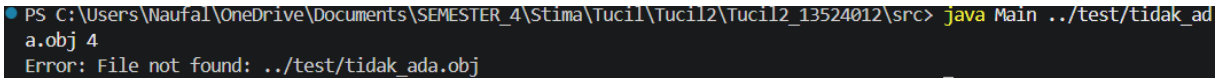


Gambar 6: Input TC01 (kiri) dan output TC01 (kanan).



```
PS C:\Users\Naufal\OneDrive\Documents\SEMESTER_4\Stima\Tucil\Tucil2\Tucil2_13524012\src> java Main ../test/cow.obj
5
[HASIL VOXELISASI]
Input file              : ../test/cow.obj
Output file             : ../test/cow_voxelized.obj
Kedalaman Maksimum      : 5
Jumlah Voxel            : 1754
Jumlah vertex terbentuk : 14032
Jumlah face terbentuk   : 21048
Kedalaman Octree        : 5
Lama waktu eksekusi     : 423 ms

Jumlah node tiap kedalaman:
0 : 1
1 : 8
2 : 32
3 : 160
4 : 728
5 : 3072

Jumlah node yang tidak diakses tiap kedalaman:
1 : 4
2 : 12
3 : 69
4 : 344
5 : 1318
```

Gambar 7: Output CLI TC01.



```
PS C:\Users\Naufal\OneDrive\Documents\SEMESTER_4\Stima\Tucil\Tucil2\Tucil2_13524012\src> java Main ../test/tidak_ad
a.obj 4
Error: File not found: ../test/tidak_ada.obj
```

Gambar 8: Output TC02.

```
PS C:\Users\Naufal\OneDrive\Documents\SEMESTER_4\Stima\Tucil\Tucil2\Tucil2_13524012\src> java Main ../test/line.obj
 abc
Error: kedalaman_maksimum harus berupa integer
```

Gambar 9: Output TC03.

```
PS C:\Users\Naufal\OneDrive\Documents\SEMESTER_4\Stima\Tucil\Tucil2\Tucil2_13524012\src> java Main ../test/line.obj
 -1
Error: kedalaman_maksimum harus >= 0
```

Gambar 10: Output TC04.

```
PS C:\Users\Naufal\OneDrive\Documents\SEMESTER_4\Stima\Tucil\Tucil2\Tucil2_13524012\src> java Main ../test/kosong.o
bj 3
Error: File is empty: ../test/kosong.obj
```

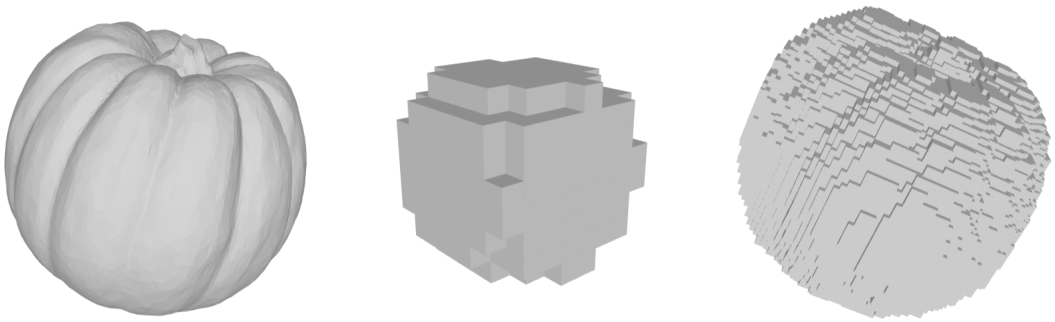
Gambar 11: Output TC05.

```
PS C:\Users\Naufal\OneDrive\Documents\SEMESTER_4\Stima\Tucil\Tucil2\Tucil2_13524012\src> java Main ../test/hanya_ve
rtex.obj 3
Error: File is empty: ../test/hanya_vertex.obj
```

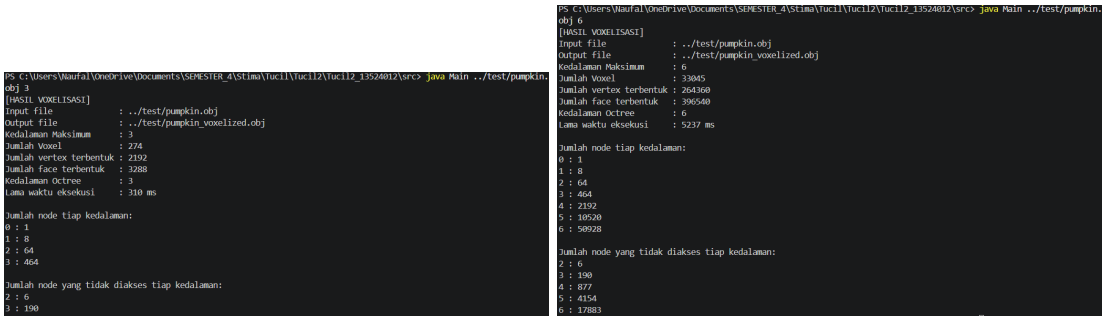
Gambar 12: Output TC06.

```
PS C:\Users\Naufal\OneDrive\Documents\SEMESTER_4\Stima\Tucil\Tucil2\Tucil2_13524012\src> java Main ../test/teapot.o
bj 3
Error: Mesh validation failed with 11 error(s):
  - Face 391: degenerate face with duplicate vertex indices (210, 211, 211)
  - Face 399: degenerate face with duplicate vertex indices (214, 211, 211)
  - Face 407: degenerate face with duplicate vertex indices (217, 211, 211)
```

Gambar 13: Output TC07.



Gambar 14: Input TC01 (kiri) output TC08 depth 3 (tengah) dan output TC08 depth 6 (kanan).



Gambar 15: Output statistik TC08 depth 3 (kiri) dan TC08 depth 6 (kanan).

5 Kesimpulan

1. Program berhasil mengimplementasikan voxelization objek 3D dari file .obj menggunakan Octree dan divide and conquer.
2. Pendekatan divide and conquer efektif karena hanya memproses bagian ruang yang relevan dengan objek.
3. Program mampu menangani berbagai kondisi input, baik valid maupun tidak valid, sesuai dengan hasil pengujian.
4. Hasil voxelization berupa kubus-kubus kecil berhasil merepresentasikan bentuk objek, dengan detail meningkat seiring bertambahnya depth.

Pustaka

[1] R. Munir, "Bahan Kuliah Strategi Algoritma," [Online]. Available: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/stima25-26.htm. Accessed: Feb. 2026.

[2] Coursera, "Data Structures and Algorithms Specialization," [Online]. Available: https://www.coursera.org/learn/algorithmic-toolbox?specialization=data-structures-algorithms. Accessed: Feb. 2026.

[3] Tim Dosen IF2211, "Spesifikasi Tugas Kecil 2 IF2211: Strategi Algoritma," Institut Teknologi Bandung, 2026.

[4] Aleksandrov, M., Zlatanova, S., and Heslop, D. J. (2021). Voxelisation algorithms and data structures: a review. Sensors, 21(24), 8241.

Lampiran

Repository Github Link: `https://github.com/tengkuNaufal/Tucil2_13524012.git`

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil di jalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	

Tabel 3: Tabel Evaluasi Program

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Tengku Naufal Saqib